

After importing the `logging` module and setting up its basic configuration, you can call `logging.debug()` to write information to a text file, as opposed to using `print()` to display it onscreen. Unlike print debugging, calling `logging.debug()` makes it obvious what output is debugging information and what output is the result of the program's normal run. You can find more information about debugging in Chapter 11 of *Automate the Boring Stuff with Python*, 2nd edition (No Starch, 2019), which you can read online at <https://autbor.com/2e/c11/>.

Variables with Numeric Suffixes

When writing programs, you might need multiple variables that store the same kind of data. In those cases, you might be tempted to reuse a variable name by adding a numeric suffix to it. For example, if you're handling a signup form that asks users to enter their password twice to prevent typos, you might store those password strings in variables named `password1` and `password2`. These numeric suffixes aren't good descriptions of what the variables contain or the differences between them. They also don't indicate how many of these variables there are: is there a `password3` or a `password4` as well? Try to create distinct names rather than lazily adding numeric suffixes. A better set of names for this password example would be `password` and `confirm_password`.

Let's look at another example: if you have a function that deals with start and destination coordinates, you might have the parameters `x1`, `y1`, `x2`, and `y2`. But the numeric suffix names don't convey as much information as the names `start_x`, `start_y`, `end_x`, and `end_y`. It's also clearer that the `start_x` and `start_y` variables are related to each other, compared to `x1` and `y1`.

If your numeric suffixes extend past 2, you might want to use a list or set data structure to store your data as a collection. For example, you could store the values of `pet1Name`, `pet2Name`, `pet3Name`, and so on in a list called `petNames`.

This code smell doesn't apply to every variable that simply ends with a number. For example, it's perfectly fine to have a variable named `enableIPv6`, because "6" is part of the "IPv6" proper name, not a numeric suffix. But if